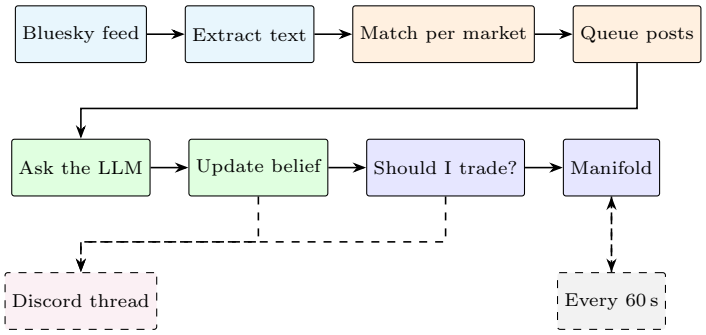
manif0ld

A Bluesky → Manifold auto-trader — how it works

One Rust binary, one tokio runtime, one SQLite database. The value is *filtering*: turn the Bluesky firehose into a handful of "this matters for market X" posts, and trade only on strong edge. Discord is the only control plane.

1 Big picture

The bot streams the Bluesky **Jetstream** firehose, cheaply filters every post through one compiled **Sieve** rule per tracked market, and only for survivors calls *one small LLM* via Cloudflare Workers AI. A material post nudges a per-market log-odds belief; the belief is compared to the live Manifold price, and a fractional-Kelly trade is placed only when the gap is wide. Each market has an isolated mana budget.



Modules: `main.rs` spawns five tasks (ingest, pipeline, trade executor, Discord, stdout logger); `db.rs` is the SQLite layer; `types.rs` holds the domain types. Text passing through `extract.rs` is never lowercased, trimmed, or NFKC-normalized — the Sieve engine does all lexical normalization internally, so a case-sensitive `"SEC"!` literal still works.

2 The per-post path

1. **Ingest** (`ingest.rs`): connect to Jetstream with `wantedCollections=app.bsky.feed.post` at the live tail; reconnect with exponential backoff (1 s → 60 s). No cursor and no replay: posts published while the socket is down are dropped, because a replayed post would move belief with news the market has already priced.
2. **Extract** (`extract.rs`): keep `commit/create` posts only; concatenate body + image alt-texts + quoted body, verbatim. Handle falls back to the author DID.
3. **Match** (`sieve.rs`): `EngineRegistry` runs one compiled Sieve per market (read-lock only).
4. **Queue** (`pipeline.rs`): each market has its own bounded triage queue (capacity 30) and one sequential worker, so belief updates stay in post order; saturation drops the oldest job.
5. **Triage** (`triage.rs`): one Workers AI call to `/ai/run` ("Authorization: Bearer" + `cf-aig-gateway-id` headers). The model returns {`material, direction, strength(0--3), rationale`}.
6. **Belief** (`belief.rs`): the post is *material* iff `material` && `direction != neutral` && `strength >= 1`; only then is belief mutated. Every triaged post is written to `signal_ledger` and emitted as a $[SIGNAL]$ event, material or not.
7. **Trade** (`trade.rs`): material signals trigger an immediate trade check for that market; a 60 s ticker re-checks all active markets (fresh price + reconciled position + decayed belief). Executed trades land in `trade_ledger` and emit $[TRADE]$.

Triage failures leave belief untouched; failed belief persistence leaves state untouched but still logs a $[ERROR]$ event.

3 Belief and position management

The bot's opinion L lives in log-odds space (clamped to $\pm 4.595 \approx \text{logit}(0.99)$). Between posts it **decays toward the market price** with a 4-hour half-life; the decay is applied *before* the new post's impulse.

$$L_{\text{dec}} = \text{logit}(p_{\text{mkt}}) + \big(L_{\text{old}} - \text{logit}(p_{\text{mkt}})\big)\, 0.5^{\,t/\tau}, \qquad \tau = 4\,\text{h}, \qquad p_{\text{int}} = \sigma(L),$$

$$\text{impulse} = \kappa \cdot \underbrace{\text{polarity}(d)}_{\substack{\text{strong_yes=1, yes=.5, neutral=0,}\\ \text{no=}-.5\text{, strong_no=}-1}} \cdot \frac{\text{strength}}{3}, \qquad \kappa = 1.1.$$

Decision (per active market). With edge $= p_{\text{int}} - p_{\text{mkt}}$ and a flat position, enter once per direction when the edge is strong:

$$\text{holding YES and edge} < 0.01 \Rightarrow \text{sell all YES}, \qquad \text{holding NO and edge} > -0.01 \Rightarrow \text{sell all NO},$$

$$f = \lambda \frac{\text{edge}}{1 - p_{\text{mkt}}} \ \text{(YES)} \quad \text{or} \quad f = \lambda \frac{|\text{edge}|}{p_{\text{mkt}}} \ \text{(NO)}, \qquad \lambda = 0.25,\ f \in [0, 0.5],$$

$$\text{target} = f \cdot \text{available}, \qquad \text{available} = \text{budget} + \text{realized PnL} - \text{invested},$$

bought only if edge ≥ 0.05 and target ≥ 1.0 mana. Sizing is fractional Kelly, exits are all-or-nothing, there is no scale-in, and the budget invariant `invested <= budget` is asserted before every buy. Position truth is Manifold's `/v0/market/{id}/positions` (reconciled every 60 s; `realized_pnl` is kept local). A global kill switch in the `kv` table blocks buys but never sells.

4 Control plane and onboarding

Discord is operator-only (one `DISCORD_OPERATOR_USER_ID`); every market gets a thread hosting its embeds.

Command	Behavior
`/track url budget?`	Onboard: fetch market, LLM synthesizes a Sieve rule, compile (retry $\leq$3), create thread, post [*ONBOARDED*] with buttons (Pause / Liquidate).
`/untrack market`	Remove engine; optionally liquidate all shares; archive thread.
`/budget market n`	Update `budget_mana`.
`/status market?`	Overview table, or per-market detail incl. Sieve source & last 10 signals.
`/override market p`	Set internal probability; triggers one trade check.
`/panic liquidate_all?`	Set the kill switch; optionally flatten everything.

Onboarding pipeline (`onboarding.rs`): parse slug from the URL, `GET /v0/slug/{slug}`, require a binary, unresolved market; ask `ONBOARDING_MODEL` for a "`match Market_<id> { ... }`" Sieve program (topic filter only — direction is never encoded in the rule; a `- @noise` exception is required); compile. "If it compiles, it's live" — the operator reviews the rule in the thread embed and can `/untrack` at any time.

Key constants

Status: spec milestones implemented; 168 unit tests green; live verification pending (model IDs, `limitProb` fills, kill-switch latency). Constants below come from `config.rs` and are overridable per market where noted.

N-per-market Sieve engines, in-process (microseconds)	matching loop
Triage queue / ingest / event channel capacities	30 / 1024 / 1024 (trade events 256)
κ (belief), `BELIEF_HALFLIFE_HOURS`	1.1, 4 h
λ Kelly fraction, `MAX_KELLY_FRACTION`	0.25, 0.5
Entry / exit edge thresholds, min trade size	0.05 / 0.01 / 1.0 mana